



Editorial

Tasks, test data and solutions were prepared by: Vito Anić, Fran Babić, Toni Brajko, Ivan Janjić, and Martina Licul.

Implementation examples are given in attached source code files.

Task Pahuljice

Prepared by: Martina Licul

Necessary skills: strings

Note that each snowflake, regardless of its size, has a '+' in the middle. Therefore, there are as many snowflakes as there are '+' characters in the drawing.

We want to determine the size of each snowflake and print the largest one.

To do so, we are going to find all the '+' characters in the drawing. For each of them, we are going to determine the size of the snowflake it belongs to. We will do that by counting how many corresponding characters there are in each of the eight directions. The size of the snowflake is the smallest of these eight numbers.

Task Pingvin

Prepared by: Marin Kišić and Martina Licul

Necessary skills: breadth-first search (bfs)

To solve the first subtask, it was enough to check a few possible cases, which we will not describe here.

To solve the second subtask, it was enough to notice that the minimum number of wing flaps needed to get from a position with x -coordinate equal to x_s to a position with x -coordinate equal to x_f is equal to the absolute difference between x_f and x_s . The same holds for the y -coordinate and the z -coordinate. The total number of wing flaps is then equal to $abs(x_f - x_s) + abs(y_f - y_s) + abs(z_f - z_s)$, where $abs(x)$ is the absolute value of the number x .

To solve the third subtask, we can omit the z -coordinate, because the penguin can only move to positions with z -coordinate equal to 1. If $z_f \neq 1$, then the answer is -1 , because the penguin cannot reach the final position.

Omitting the z -coordinate, we have reduced the three-dimensional space to a two-dimensional plane. The algorithm that solves this task is *breadth-first search*. This algorithm starts from the initial position of the penguin and in each step it expands to all neighboring positions, checking whether it has already been to that position and whether it can be at that position. If both conditions are satisfied, then that position is added to the queue, and the number of wing flaps needed for that position is equal to the number of wing flaps needed for the position from which it came, increased by 1. After a position is processed, it is removed from the queue, and the algorithm continues with the processing of the next position in the queue. The algorithm stops when the final position is processed or when the queue is empty.

To solve the whole task, it is necessary to extend the algorithm from the previous subtask to a three-dimensional space. For implementation details, see the attached code. For more information about the *breadth-first search* algorithm, see the [link](#).

Task Dizalo

Prepared by: Vito Anić

Necessary skills: Fenwick tree or Segment tree

In this editorial, if we call someone person x , we mean the person who wants to leave at floor x .



If a person wants to get out of the elevator and in front of them there are other people, they will have to go out too. But the best way they can return to the elevator is that they are sorted by the floor on which they want to exit. The closest to the door would be someone who wants to exit soonest.

Let us now solve subtask $q = 0$.

Observe that we can split people into two groups. People who will exit the elevator just once, and people who will exit the elevator multiple times. For example: 4 5 2 1 6 3 7: people 1, 3, and 7 will exit the elevator once, and the rest of them (4, 5, 2, 6) will exit multiple times. We can notice that if someone is the minimum in its suffix, they will exit the elevator only once. To clarify, if one does not have a person who wants to go out before him behind him, there would be no reason for him to exit before he wants to. Because of the way we are retributing people (sorting them) if someone is exiting the elevator multiple times, on the last exit, he will be able to leave without anybody having to exit with him.

Let us now look at the people who exit only once. With them, everyone who wants to leave on a later floor and is in front of them will have to exit when they leave. That can be calculated with Segment or Fenwick tree. When we know that for everyone, the solution will be n plus that sum.

In the example above, because of 1, persons 4, 5 and 2 are forced to exit. Because of 3, persons 4, 5 and 6, and nobody because of 7. The solution is $3 + 3 + 0 + 7 = 13$.

Now we know how to solve $q = 0$. There is one simplification of the calculation above. For calculating the number of people in front of them and leaving on a later floor there is a very nice formula: *index - value* (index is the position in the elevator and value is the floor on which they want to exit). That formula **only** works for permutations! To clarify why is that correct, if a number is smallest in its suffix, then every number smaller than him is in front of him, which means that there are value - 1 smaller numbers in front of him, and the rest are larger.

For the full solution of this task, we need to maintain the property that people in the elevator form a permutation, and somehow be able to the set of people who exit the elevator only once. For the former when erasing some element, using Segment or Fenwick trees we can maintain the values and indices. When erasing a person who exits multiple times, we must reduce the solution by the number of people who exit once are behind them and exit at a smaller floor. When erasing a person who exits the elevator only once, more people can enter that set of people. The easiest way to maintain that set is to calculate offline for every person how long will they be in the set. For detailed implementation look at the attached file. Total time complexity is $\mathcal{O}(n \cdot \log n)$.

Task Kuglice

Prepared by: Fran Babić

Necessary skills: basic game theory, dynamic programming

In the first subtask, it holds that there are at most two different colors in the sequence. The first ball taken will definitely score 1 point (for Marin), so the only possible outcomes are 1:0, 1:1, and 2:0. Solving the first subtask boils down to decomposing it into 3 different cases:

1. All elements in the sequence are the same, i.e., the number of different colors is one.
In this case, the game result will be 1:0.
2. The element at the left end of the sequence is different from the one at the right end, i.e., $a_1 \neq a_n$.
Whichever element Marin chooses in the first move, Josip can take the one from the other end of the sequence, which is different from the one Marin took. We can see that in this case, the game result will be 1:1.
3. Not all elements in the sequence are the same, and the elements at the ends of the sequence are the same ($a_1 = a_n$).
It is easy to see that the answer will depend on the parity of the longest prefix and suffix of equal elements.



Let $L_i = \min_{a_j=i} j$ and $R_i = \max_{a_j=i} j$, i.e., the index of the first occurrence and the index of the last occurrence of a color i in the sequence. Consider some game state (l, r) where l is the index of the leftmost ball in the box, and r is the index of the rightmost ball in the box. For a complete solution, we need the following observation:

Observation. *For the current game state (l, r) , a ball of color i has been taken in some previous moves if and only if $L_i < l$ or $R_i > r$.*

Using this, we can write a function $f(l, r)$ that returns the difference in points between the first and second player in that game state. The value returned by $f(l, r)$ depends on the values of $f(l + 1, r)$ and $f(l, r - 1)$. The complexity of such an algorithm is $O(2^n)$, but by memoizing the function, we reduce the complexity to $O(n^2)$, which is sufficient to score all of the points. Details of the implementation can be seen in the official source code.

Task Zatopljenje

Prepared by: Ivan Janjić

Necessary skills: Segment tree

Let's look at the definition of an island. It is an interval, thus it is defined with positions of its left and right border. Maximality ensures that immediately to the left and right is sea or a border of a query. It's not hard to prove that two islands have no borders in common and that every island has exactly two borders. Therefore, to count islands, it's sufficient to count borders and divide the number by 2. Let's observe a single query. Let's imagine an interval $[l, r]$ as a sequence of zeroes and ones, where 1 means relief is higher than the sea level at that position, and 0 otherwise. Add zeroes to the beginning and end of this sequence. Now the task is to count how many times do situations 01 and 10 occur.

To solve the first subtask, it was enough to do exactly what was described above. Overall complexity of this approach is $\mathcal{O}(n \cdot q)$

To solve the second subtask, we use the fact that we are given queries in advance and that the mentioned sequence does not change much when going from one interesting sea level to the next interesting sea level with lower value (Interesting sea levels are a set that contains heights of all positions in a relief and all heights encountered in queries). More precisely, some zeros get turned to ones. Turning a 0 to 1 can only affect 2 borders. As all queries ask for number of islands for the whole relief, it was enough to maintain for every boundary between positions whether it is a border of an island or not, update the total amount accordingly and answer a query when sea level is exactly the one requested in the query. Overall complexity of this approach is $\mathcal{O}(n + q)$

To solve the third subtask, it was enough to observe that, given the relief structure, there can be no more than two islands in each query. What's left is to determine the conditions for when there are 0, 1 or 2 islands. Details are left as an exercise to the reader. Overall complexity of this approach is $\mathcal{O}(n + q)$

We use the idea for the solution of the second subtask to solve the full problem. When looking at the sea at some particular level, we need to know how many boundaries are island borders in an arbitrary interval. When changing the sea level, we need to update some boundaries. We need to do the first operation q times, and the second $\mathcal{O}(n)$ times overall, as we will update every position exactly once. These operations can be implemented with a segment tree built on the boundaries between positions. Overall complexity of this approach is $\mathcal{O}((n + q) \cdot \log n)$. For details of implementation refer to the official solution.